

User-defined erroneous behaviour

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Motivation](#)
4. [Proposal: `std::erroneous`](#)
5. [How to use `std::erroneous`](#)
6. [Impact and implementability](#)
7. [Proposed wording](#)
8. [Acknowledgements](#)
9. [References](#)

Revision history

- [P3232R0](#): Initial revision.
- P3232R1: This revision; adds further motivation

Summary

We propose a language-support library function that has no effect other than to cause erroneous behaviour. This allows user-defined APIs to include erroneous behaviour.

Motivation

The purpose of introducing the novel erroneous behaviour in [P2795R5](#) was to add to C++ the ability to express operations that have well-defined behaviour in the presence of certain programming errors, thereby mitigating the safety and security implications of said errors: Erroneous behaviour is well-defined and part of the observable behaviour of a program, and the compiler must not assume that it does not happen (unlike undefined behaviour).

Recall the guiding observation:

Erroneous behaviour is well-defined behaviour.

P2795R5 only prescribed erroneous behaviour for a particular operation (namely reading an uninitialized variable with automatic storage duration).

But being able to declare parts of an API erroneous is also useful for user-defined APIs that have preconditions (i.e. a “narrow contract”). It is a programming error to invoke such an API when preconditions are not met, but currently, attempts to make this API “safe”, that is, to limit the damage caused by calling it out of contract, suffer various shortcomings:

- We can leave the behaviour out of contract undefined. This is easy to specify and follows the Standard Library convention, but has poor safety implications, as basically anything could happen. The library implementer is not constrained in their implementation, which may or may not encounter actual undefined behaviour, or take some reasonable precautions.
- We can fully specify the behaviour out of contract (e.g. terminating or throwing an exception). This makes the API technically have a wide contract (as there are no technical preconditions), and the intended preconditions have to be communicated out-of-band, e.g. just in natural language.
- We can check the preconditions in a subset of cases (e.g. with `assert` when `NDEBUG` is not defined. This retains the narrow contract, but leaves the safety implications somewhat opaque.
- A hypothetical "contracts" facility in the language (e.g. see [P2900R10](#) for a recent iteration) would allow a programmatic statement of the precondition, but there is not yet an agreed-upon definition of the precise semantics.

With erroneous behaviour, we can get the best of both worlds: we can keep the narrow contract, still have well-defined, i.e. “safe”, behaviour in case of violation (because remember, erroneous behaviour is well-defined behaviour), but we make it unambiguously clear that a precondition violation is a programming error. Production code will not misbehave arbitrarily, but instead behave predictably, and debugging tools can actually diagnose the programming error confidently.

To illustrate this on a simple example, let us consider a small function that computes the quotient of two floating point numbers and has the precondition that the denominator not be zero:

Undefined behaviour

```
// Precondition: `den` must not be zero
float quotient(float num, float den) {
    return num / den;
}
```

Checked with **assert**

```
// Precondition: `den` must not be zero
float quotient(float num, float den) {
    assert(den != 0);
    return num / den;
}
```

Well-defined violation

```
// Precondition: `den` must not be zero,
// terminates otherwise
float quotient(float num, float den) {
    if (den == 0) { std::abort(); }
    return num / den;
}
```

With a “contracts”
facility,
precondition:

```
// Precondition and contract: `den` must
not be zero.
float quotient(float num, float den)
pre(den != 0) {
    // Option #1: undefined behaviour on
violation
    /* nothing */

    // Option #2: well-defined behaviour on
violation
    if (den == 0) { return -2; }

    return num / den;
}
```

With a “contracts”
facility,
contract_assert:

```
// Precondition: `den` must not be zero.
float quotient(float num, float den) {
    contract_assert(den != 0);

    // Option #1: undefined behaviour on
violation
    /* nothing */

    // Option #2: well-defined behaviour on
```

```
violation
    if (den == 0) { return -2; }

    return num / den;
}
```

```
// Returns the quotient `num`/`den`;
// if `den` is zero, returns -2
// erroneously.
float quotient(float num, float den) {
    if (den == 0) { std::erroneous();
return -2; }
    return quotient(unsafe_unchecked, num,
den);
}
```

```
// As above, but precondition violation
// is undefined
float quotient(unsafe_unchecked_t, float
num, float den) {
    return num / den;
}
```

**Proposal: violation is
erroneous**

The final row demonstrates the proposed feature: by allowing user-defined code to be erroneous, we can offer a safe-by-default API that has just the same preconditions as an unsafe API would have had, but with well-defined behaviour (or termination; see P2795R5) in case the user makes a mistake. The original, unchecked API can be provided as a separate, explicitly annotated overload.

Proposal: **std::erroneous**

We propose a language-support function **std::erroneous** that has no effect other than to have erroneous behaviour.

The proposed function is to **std::unreachable** as erroneous behaviour is to undefined behaviour:

Function	Behaviour if invoked
std::unreachable	undefined behaviour

Function	Behaviour if invoked
<code>std::erroneous</code>	erroneous behaviour; no effect

How to use `std::erroneous`

Step 1: Identify an opportunity. Erroneous behaviour could be used in operations that have preconditions. To use erroneous behaviour, two conditions have to be met:

- The precondition must be testable programmatically by the called code.
- You have to have some implementable behaviour in mind that the operation will exhibit when the precondition is not met. This could be something like producing some fixed value, some fixed control flow, throwing an exception, or terminating.

Step 2: Consider API alternatives. Consider how you would like the operation to incorporate the precondition. Is it definitely always a user error for the precondition to be violated, or should the operation handle the precondition violation in a specified way that a user is allowed to use and depend on? In the latter case, the precondition actually becomes part of the normal operation, and the normal operation becomes more complex. In the former case, we tell users “not to do that”, and we can use erroneous behaviour to safeguard against misuse.

Step 3: Create an opt-out. If you place the operation with precondition into a separate function, which should be clearly labelled as something like “unsafe” or “unchecked”, then callers who are certain that the preconditions are met can choose to call this implementation and not incur any performance penalty for the precondition check. The main, safe function can then be implemented in terms of the unsafe one.

Step 4: Implement. To make the operation with preconditions use erroneous behaviour, check the condition, and if it does not hold, call `std::erroneous` and then perform the fallback behaviour identified in Step 1.

```
if (precondition) {
    /* call unsafe implementation */
} else {
    std::erroneous();
    /* fallback behaviour */
}
```

In fact, let us have a concrete example and consider a function with preconditions. We have already renamed the function with the “Unchecked” suffix in anticipation of the next step.

```
// Requires: request_type must be either kType1 or kType2
std::uint64_t FetchRequestKeyUnchecked(int request_type,
std::string_view user_name);
```

It is always a user error to pass an invalid `request_type`. We could widen the contract and specify a return value, but that would complicate the API only to allow something that is not useful in the first place (see Step 2). But if we decide to keep the API narrow, we need to pick a behaviour: terminating (with some form of debug assertion) is a plausible option, or perhaps we have an invalid value that we know will be rejected elsewhere in the system and that we can return here, erroneously:

```
// Requires: request_type must be either kType1 or kType2;
// violation is erroneous behaviour and results in an invalid
value.
std::uint64_t FetchRequestKey(int request_type, std::string_view
user_name) {
    switch (result_type) {
        case kType1:
        case kType2:
            return FetchRequestKeyUnchecked(request_type, user_name);
        default:
            std::erroneous();
            return kInvalidKey;
    }
}
```

Effects.

- The operation is safe in the sense that precondition violation does not result in undefined behaviour. It either behaves as specified, or is diagnosed by a suitable tool. Practically, the operation cannot run into problematic compiler optimisations, since the compiler cannot assume that the erroneous behaviour does not happen. This is the main distinction from leaving the case entirely unhandled, and effectively telling the user that the behaviour is undefined.
- The programming error is detectable systematically by appropriate tools such as runtime sanitizers, since erroneous behaviour is allowed to be diagnosed. The manner of diagnosis is up to the tool; e.g. it is common to terminate on the first detected occurrence, or to continue and collect multiple detections.
- A “production” platform would likely just perform the specified fallback behaviour (see also the [P2795R5 Tooling](#) section on suggested platform profiles). A high-performance, safety-uncritical platform could *assume* that the erroneous behaviour is not reached. On neither platform would

calling `std::erroneous` cause termination, no more than reading an uninitialized variable would.

Impact and implementability

Platforms that diagnose erroneous behaviour will presumably provide some builtin hook with which the feature can be implemented. Otherwise, it is always conforming to implement this feature as a no-op.

Alternatives

An `[[erroneous]]` attribute was suggested during informal discussions, but it does not seem compelling: For example, it does not seem useful to annotate an entire function as always having erroneous behaviour. Instead, it is more composable to express this concern separately and once and for all, namely in the proposed function `std::erroneous`.

Proposed wording

In either [support] or [diagnostics], in a header to be determined, add a new function:

```
// User-defined erroneous behavior  
void erroneous();
```

Add the specification:

User-defined erroneous behavior

```
void erroneous();
```

Effects: The behavior is erroneous; calling this function has no effect otherwise.

Acknowledgements

Many thanks to Barry Revzin and Oliver Hunt for helpful questions and discussion, to Alisdair Meredith for review, and to members of SG21 for feedback on the connection to contracts.

References

- Thomas Köppe, [P2795R5](#): *Erroneous behaviour for uninitialized reads*.
- Thomas Köppe, [N5001](#): *Working Draft, Programming Languages — C++*.
- Joshua Berne et al., [P2900R10](#): *Contracts for C++*.